Chat Sync Fixes and Multi-Device Key Plan

Chat Sync Fixes and Multi-Device Key Plan

Date: 2026-05-25

Summary

This document records the first-pass fixes made to improve Qbase chat/contact synchronization and message queue reliability. The immediate problem was that contact names and public keys were not always available locally when messages were sent or retried, causing encrypted messages to remain queued. Realtime message and chat events were also subscribed through an outdated Appwrite channel format while the app now writes through TablesDB.

The implemented fixes target the highest-impact causes:

- Realtime chat/message subscriptions now listen to Appwrite TablesDB row channels.
- Reconnect sync now refreshes chat metadata and participant profiles before flushing queued messages.
- Pending message retry now refreshes all participants' profiles/public keys before each send attempt.
- Friend-code search no longer trusts stale local contact rows when public keys or names are incomplete.

The deeper multi-device encryption issue is not solved in this pass. A migration plan is included later in this document.

Root Causes Identified

1. Realtime Channel Mismatch

The app writes data through Appwrite TablesDB APIs:

- `tablesDB.upsertRow`
- `tablesDB.getRow`
- `tablesDB.listRows`

However, chat/message realtime observers were still subscribed to old database document channels:

- `databases.qbase_db.collections.messages.documents`
- `databases.qbase_db.collections.chats.documents`

Because of that mismatch, realtime chat and message events could fail to arrive even though REST-style sync still worked.

2. Queue Flush Happened Before Contact/Key Sync

On reconnect, the previous sequence flushed pending messages before syncing remote chats and participant profiles. Encrypted sends require local participant public keys. If the local contact cache was empty or stale, queued messages retried too early and stayed pending.

3. Queued Messages Did Not Self-Heal

`flushQueue()` retried pending messages but did not proactively refresh the participants for each message's chat. A message that failed due to missing keys could keep retrying with the same missing local data.

4. Local Contact Cache Could Be Stale

Friend-code search checked Room first and returned a cached contact immediately. If that cached contact had a blank public key, placeholder display name, or outdated profile data, chat creation and encryption inherited the stale row.

5. One Public Key Per User Breaks Multi-Device E2EE

The current profile model stores one `publicKey` on the user row. When a user signs in on another device, that device can overwrite the user's public key. Other users may then encrypt future messages to the new device key, while the old device cannot decrypt them.

Implemented Fixes

1. TablesDB Realtime Channels

Updated realtime subscriptions to match the TablesDB storage layer:

- Messages: `tablesdb.qbase_db.tables.messages.rows`
- Chats: `tablesdb.qbase_db.tables.chats.rows`

Files changed:

- `app/src/main/java/com/algorithmx/q_base/sync/orchestration/MessageSyncIncomingExtensions.kt`
- `app/src/main/java/com/algorithmx/q_base/sync/orchestration/Messag`

`eSyncIncomingGlobalExtensions.kt`

Expected impact:

- Incoming messages should arrive more reliably in realtime.
- Chat create/update/delete events should sync through the active Appwrite event namespace.

2. Reconnect Sync Order

Changed reconnect flow to sync data before flushing queued encrypted messages:

1. Sync current user profile.
2. Sync remote chats.
3. Sync participant profiles as part of chat sync.
4. Flush pending message queue.
5. Flush universal action queue.

File changed:

- `app/src/main/java/com/algorithmx/q_base/MainActivity.kt`

Expected impact:

- Pending messages have a better chance of finding participant public keys before retrying.
- The queue should no longer fail immediately just because chat/contact sync had not run yet.

3. Participant Profile Refresh During Queue Flush

Added a per-message refresh before each queued send attempt. For every pending message, the app now:

1. Loads the message's chat.
2. Reads chat participant IDs.
3. Syncs those participant profiles.
4. Attempts encrypted send.

File changed:

- `app/src/main/java/com/algorithmx/q_base/sync/orchestration/MessageSyncOutgoingExtensions.kt`

Expected impact:

- Queued messages become more self-healing.
- Missing public keys can be fetched immediately before retry instead of waiting for a separate sync path.

4. Stale Local Contact Refresh

Updated friend-code search so local cached contacts are refreshed before being returned when they look incomplete or placeholder-like.

A cached contact now refreshes if:

- `publicKey` is blank.
- `displayName` is blank.
- `displayName` is `Learner`.
- `displayName` is `Knowledge Seeker`.
- `displayName` starts with `User`.

File changed:

- `app/src/main/java/com/algorithmx/q_base/feature/chat/presentation/ContactSelectorViewModel.kt`

Expected impact:

- Starting a chat from a stale local contact is less likely to create a conversation with a missing public key.
- Contact names should refresh more often when old placeholder values are present.

Verification

The following check passed:

```
./gradlew :app:compileDebugKotlin
```

Result:

```
BUILD SUCCESSFUL
```

Also verified that no old chat/message realtime channels remained in app/core source:

```
rg -n "databases\.qbase_db\.collections\.(messages|chats)\.documents|tablesdb\.qbase_db\.tables\.(messages|chats)\.rows" app core-* -S
```

Only the new TablesDB chat/message row channels remained.

Remaining Risks

Multi-Device E2EE

The app still stores a single public key per user. That means multi-device behavior can still fail:

1. User signs in on phone and publishes phone public key.
2. User signs in on tablet and publishes tablet public key.
3. Sender encrypts future messages to tablet public key.
4. Phone receives those messages but cannot decrypt them.

This is a protocol/data-model issue, not just a sync timing bug.

Stale But Valid Keys

The current first-pass fix refreshes missing/incomplete keys more aggressively, but it does not prove a cached non-empty key is the latest intended key. A stale but valid key can still be used until the multi-device key model is implemented.

Queue Failure Visibility

Pending messages still do not store a structured failure reason. The queue logs failures, but the local message status model should eventually distinguish:

- `PENDING`
- `SENT`
- `FAILED_MISSING_KEY`
- `FAILED_UNREADABLE_PROFILE`
- `FAILED_NETWORK`

This would make debugging and UI feedback clearer.

Multi-Device Key Plan For Later Implementation

Goal

Replace the single `users.publicKey` model with a per-device public key registry so encrypted messages can be delivered to every active device owned by each recipient.

Proposed Schema

Keep the `users` table for public profile data:

```
users
  userId
  displayName
  profilePictureUrl
  friendCode
  intro
  isBanned
  isPhotoVisible
```

Add a device key table:

```
user_devices
  id: "{userId}_{deviceId}"
  userId: String
  deviceId: String
  deviceName: String?
```

```
publicKey: String
keyFingerprint: String
createdAt: Long
lastSeenAt: Long
revokedAt: Long?
```

Recommended indexes:

- `userId`
- `userId + revokedAt`
- `keyFingerprint`

Permissions:

- Read: authenticated users, because senders need recipient public keys.
- Create/update/delete: only the owner user.

Device Identity

Each install should generate a stable local `deviceId` and store it in encrypted/local preferences. The private key remains local to that device unless secure backup/restore is explicitly used.

On login/session restore:

1. Initialize or restore local E2EE private key.
2. Derive/export the public key.
3. Register or update `user_devices/{userId}_{deviceId}`.
4. Update `lastSeenAt`.

Sending Messages

For each message:

1. Resolve chat participants.
2. Query active devices for each participant.
3. Encrypt one message payload with a fresh symmetric session key.
4. Wrap that session key once per recipient device public key.
5. Store wrapped keys by device, not by user.

Example wrapped key payload:

```
{
  "recipientUserId": {
    "deviceIdA": "wrapped-session-key",
    "deviceIdB": "wrapped-session-key"
  }
}
```

Alternative flattened form:

```
{
  "userId:deviceIdA": "wrapped-session-key",
  "userId:deviceIdB": "wrapped-session-key"
}
```

The flattened form is simpler to query and deserialize, but the nested form is clearer.

Receiving Messages

On receive:

1. Load current local `deviceId`.
2. Look for `wrappedKeys[currentUserId][deviceId]` or `wrappedKeys["$userId:$deviceId"]`.
3. Decrypt the session key with the local private key.
4. Decrypt the message payload.

If no wrapped key exists for this device, mark the message as not decryptable on this device instead of treating it as a generic decryption failure.

Migration Strategy

Phase 1: Compatibility

- Keep reading `users.publicKey`.
- Start writing `user_devices`.
- New sends prefer `user_devices`.
- If no devices exist for a recipient, fall back to `users.publicKey`.

Phase 2: Backfill

- On each app launch, current users register their current device key.
- Friend/contact sync fetches device keys when available.
- Add telemetry/logging for fallback sends.

Phase 3: Enforce Device Keys

- Stop writing `users.publicKey` for new key material.
- Keep `users.publicKey` only as a legacy fallback field.
- Add UI messaging when a recipient has no registered device key.

Phase 4: Cleanup

- Remove fallback logic after enough versions have migrated.
- Remove or ignore `users.publicKey`.

Revocation

Users should be able to revoke devices. A revoked device:

- Remains in history for audit/debug visibility.
- Is excluded from future sends.
- Cannot receive newly wrapped session keys.

Existing messages are not re-encrypted during revocation.

Testing Checklist

- One user, one device: send/receive still works.
- Two users, one device each: P2P chat works.
- Recipient has two devices: both devices decrypt new messages.
- Sender has two devices: both sender devices can decrypt their own sent messages only if sender devices are included in wrapped keys.
- Revoked device: no new wrapped keys are generated for it.
- Offline queue: pending messages resolve device keys before retry.
- Legacy recipient without device rows: fallback to `users.publicKey` works during migration.

Recommended Next Work

1. Add structured pending-message failure statuses.
2. Add tests around reconnect queue flush order and missing-key recovery.
3. Implement the `user_devices` table and compatibility write path.
4. Move outgoing encryption from per-user wrapping to per-device wrapping.
5. Add device revocation UI and cleanup policy.